

Deployment Manual — Zoho Mail

Setting up the hiring dashboard from nothing: a spreadsheet, a robot Google account, a Zoho app password, and one deploy. Six phases, about fifty minutes. The code is identical to the Gmail build — Zoho is four environment variables.

Company domain already on Zoho

No card required

No code changes

PHASE 0 What you are building

One Next.js service on Render, talking to three outside things.

SERVICE	ROLE	COST
Google Sheet	The database — candidates, templates, settings, log	Free
Groq	Writes the emails	Free tier
Zoho Mail SMTP	Sends them	Included with the mailbox; daily cap depends on plan

There is no separate backend and no other database. The sheet *is* the database — you type into it, the app reads and writes back to it.

PHASE 1 The spreadsheet

10 min

1.1 Create it

New Google Sheet, named anything. Look at the URL and copy the part between `/d/` and `/edit` — that is `SHEET_ID`.

```
https://docs.google.com/spreadsheets/d/1a2B3cD4eF5gH6iJ7kL8mN9oP/edit
                                └──────────────────┘
                                SHEET_ID
```

1.2 Service account

A **service account** is a robot Google account. Unlike a normal login it has no consent screen, no browser popup, and no token that expires — it is a key that works until you revoke it. That is why the project uses one.

At console.cloud.google.com (<https://console.cloud.google.com>):

1. **Create a project** (top-left dropdown → New Project). Name it `hr-automation`.
2. **APIs & Services → Library** → search "Google Sheets API" → **Enable**. Miss this and every call fails with a 403.
3. **IAM & Admin → Service Accounts → Create service account**. Name it `hr-dashboard`. Skip the optional roles and access steps — they do not apply here.
4. Open it → **Keys** → **Add key** → **Create new key** → **JSON**. A `.json` file downloads.

That file is a password. Never commit it, never paste it into a chat or a ticket. It has full write access to every sheet you share with it.

1.3 Share the sheet with the robot

Open the downloaded JSON and find `client_email`:

```
"client_email": "hr-dashboard@hr-automation-412....iam.gserviceaccount.com"
```

Copy that address. In the spreadsheet → **Share** → paste it → **Editor** → Send.

This is the step everyone skips. A service account cannot see a sheet just because you own it — you share it explicitly, exactly as you would with a person. Missing this produces `E-SHEET-PERM`.

PHASE 2 Zoho app password

5 min

A Zoho **app password** is a per-application password that logs into Zoho's SMTP server directly. The dashboard authenticates with it exactly as it would with Gmail — no code path changes.

1. Sign in to the sending mailbox at `mail.zoho.com` (<https://mail.zoho.com>) (or `mail.zoho.in` (<https://mail.zoho.in>) for India-hosted accounts).
2. Turn on multi-factor auth: **Settings** → **Security** → **Multi-Factor Authentication** → **enable**. Without it, Zoho will not create an app password.
3. Go to `accounts.zoho.com` (<https://accounts.zoho.com>) → **Security** → **App Passwords** → **Generate New Password**. Name it `hr-dashboard` → **Generate**.
4. Copy the string it shows and paste it into `MAIL_PASSWORD` **without spaces**. It is shown once.

2.1 Confirm the SMTP host

In Zoho Mail → **Settings** → **Mail Accounts** → click the address → the **IMAP/SMTP** (Outgoing Server) section shows the exact host for your account. It must match the region your Zoho account is hosted in:

ACCOUNT REGION	MAIL_HOST
US / global	<code>smtp.zoho.com</code>
India	<code>smtp.zoho.in</code>
Europe	<code>smtp.zoho.eu</code>
Australia	<code>smtp.zoho.com.au</code>
Paid organisation (Zoho Workplace / Mail paid)	<code>smtppro.zoho.com</code> (regional variants exist)

Use `MAIL_PORT=465` (implicit TLS). Port 587 also works (STARTTLS) but 465 is the default the code assumes.

Wrong region = silent failure. Pointing `MAIL_HOST` at `smtp.zoho.com` when the mailbox lives on `smtp.zoho.in` fails with `E-MAIL-AUTH` or hangs as `E-MAIL-NETWORK`. Check the SMTP section in Settings rather than guessing.

Check the plan allows SMTP. Zoho's Forever-Free plan restricts IMAP/SMTP access on some tiers, and every plan has a daily sending limit (roughly 250/day for a new account, more once the account is established and on a paid plan). Confirm SMTP is enabled and note the limit — you set `send_daily_cap` under it in Phase 4.

An app password grants full access to that mailbox over SMTP and IMAP — send *and* read. It is revocable in one click from the same App Passwords page, it is named so its owner can see what it belongs to, and every send it makes lands in that mailbox's Sent folder — a free second audit trail. Because your company owns the domain, Zoho keeps `MAIL_FROM` exactly as you set it.

2.2 Groq key

`console.groq.com/keys` (<https://console.groq.com/keys>) → sign in → **Create API Key**. Free tier, no card.

PHASE 3 Run it locally first

10 min

Do this before deploying. Debugging on your own machine is far faster than debugging through build logs.

```
npm install # root: the sheet setup scripts
cd dashboard && npm install # the web app itself
```

Two `package.json` files, two installs — the dashboard deploys from `dashboard/` alone and cannot reach outside that folder, so it carries its own dependencies.

3.1 The env file

```
cp .env.zoho.example .env.local # you are in dashboard/
```

Edit `.env.local`, never `.env.zoho.example`. The template is tracked by git; the `.local` file is ignored. Values typed into the template are one `git commit -a` away from being published, and a leaked service-account key must then be revoked and reissued. Nothing reads `.env.zoho.example` at runtime, so the symptom is a confusing "SHEET_ID is not set" on a file that visibly has one.

```
SHEET_ID=1a2B3cD4eF5gH6iJ7kL8mN9oP
GOOGLE_SERVICE_ACCOUNT_JSON='{ "type": "service_account", "project_id": ... }'

DASHBOARD_PASSWORD=shared-with-the-hr-team
SESSION_SECRET=<output of: openssl rand -hex 32>

GROQ_API_KEY=gsk...

MAIL_USER=hiring@yourcompany.com
MAIL_PASSWORD= # ← LEAVE BLANK for now, on purpose
MAIL_FROM=3Space Hiring <hiring@yourcompany.com>
MAIL_HOST=smtp.zoho.in # ← the host you confirmed in step 2.1
MAIL_PORT=465
```

- `GOOGLE_SERVICE_ACCOUNT_JSON` — flatten the downloaded JSON to **one line** and wrap it in **single** quotes, because the JSON itself is full of double quotes.
- `SESSION_SECRET` — signs the login cookie. Any 64 random hex characters; it only has to be unguessable.
- `MAIL_PASSWORD` — **leave blank deliberately**. The app refuses to send without it, which is what you want while still clicking around.
- `MAIL_HOST` / `MAIL_PORT` — **required for Zoho**. Unset, they default to `smtp.gmail.com:465` and every send fails. Set the host to the regional value from step 2.1.

3.2 Build the tabs

```
cd .. && npm run bootstrap:sheets
```

Creates all four tabs with the right headers and seeds the Config defaults. **Idempotent** — safe to re-run any time; it repairs rather than duplicates, and it is the fix for `E-SHEET-SCHEMA` .

```
• tab "Applicants" created
• tab "Templates" created
• tab "EmailLog" created
• tab "Config" created
• "Applicants" headers written (15 columns)
• Config key "dry_run" added (= true)
...
✓ Done.
```

Optional, and worth it the first time:

```
npm run seed:demo    # one working template + 3 fake candidates
```

3.3 Start it

```
cd dashboard && npm run dev    # http://localhost:3000
```

Log in with `DASHBOARD_PASSWORD` . You should see the **Inbox** with the seeded candidates.

With `SHEET_ID` left blank the app runs on a built-in sample dataset instead — fully clickable, nothing saved, nothing sent. Useful for a look around before committing to any Google setup.

PHASE 4 The sheet's shape

You type into five columns. Everything to the right of `stage` is written by the app — leave those cells empty.

APPLICANTS

A	B	C	D	E	F
applicant_id	name	email	job_role	category	notes
APP-1001	Priya Shah	priya@...	Frontend Intern	Intern	met at the PDEU fair, strong React

Then `stage` in column G, and eight app-written columns after it.

- **applicant_id must be unique.** Every action resolves a row by it, and a lookup returns the *first* match — so a duplicate id means acting on the second row silently acts on the first. The Inbox flags these with a banner naming the sheet rows, and preflight fails on them. Any format works: `APP-1001`, `1`, `priya-shah`.
- **notes is the highest-leverage column.** Free text handed to the model as context. "Met at PDEU fair, strong on React, weak on backend" produces a specific email; an empty cell produces a generic one.

TEMPLATES

Fill `subject` and `html` with merge fields — `{{name}}`, `{{job_role}}`, `{{company_name}}`, `{{hr_signature}}`. Set `is_active` to `TRUE` for a template to be usable, and `is_default` to `TRUE` on exactly one.

Merge fields are **fail-closed**: an unresolved `{{field}}` blocks the send rather than mailing someone the literal text.

CONFIG

Key/value settings, edited on the app's **Settings** page rather than by hand.

KEY	SHIP AS	MEANING
<code>dry_run</code>	<code>true</code>	Log sends instead of delivering
<code>toggle_send</code>	<code>false</code>	Master switch for sending
<code>send_daily_cap</code>	under your Zoho plan's limit	Self-imposed ceiling. Set it below the daily send limit for your Zoho plan (check Zoho Mail Admin → Mail limits)
<code>company_email</code>	your address	Becomes the <code>Reply-To</code>

5.1 Push

```
git push origin main
```

The blueprint tracks `main` (`render.yaml`, `branch: main`). If the code currently lives on another branch, merge it to `main` first or change the branch in the Render service settings. No source change is needed for Zoho — only the environment variables in step 5.3 differ.

5.2 Blueprint

dashboard.render.com (<https://dashboard.render.com>) → **New** → **Blueprint** → connect GitHub → pick the repo.

Render reads `render.yaml`, creates one web service, and prompts for each secret. That file already sets the build command, the start command, Node 22, and a health check on `/login` — not `/`, which redirects when signed out and would read as a 307 failure.

5.3 Fill in the secrets

Same values as `.env.local`, except paste the service-account JSON raw (no quotes).

VARIABLE	VALUE
DASHBOARD_PASSWORD	same
SESSION_SECRET	same
SHEET_ID	same
GOOGLE_SERVICE_ACCOUNT_JSON	Paste the raw JSON — no quotes, no escaping. Render's value box is not a shell, so the single quotes needed locally would be read as part of the JSON and break it.
GROQ_API_KEY	same
MAIL_USER	the full Zoho sending address
MAIL_PASSWORD	now paste the Zoho app password, no spaces
MAIL_FROM	display name and address
MAIL_HOST	the regional host from step 2.1, e.g. <code>smtp.zoho.in</code>
MAIL_PORT	465

`render.yaml` already lists `MAIL_HOST` and `MAIL_PORT` as promptable secrets, so the Blueprint will ask for them. Leave only `GROQ_MODEL` blank — it defaults to `llama-3.1-8b-instant`. First build takes about four minutes.

The free plan sleeps after 15 minutes idle, so the first request after a quiet spell takes ~30 seconds. Fine for an internal tool.

PHASE 6 Verify, then go live

Do not reorder this. Every step is a chance to catch a problem while it is still harmless.

6.1 Preflight

Open the deployed URL → log in → **Console** → **Run preflight**. Every check must pass: sheet reachable, headers matching the schema, Groq responding, no duplicate ids, no malformed addresses, and the mailbox actually logging in.

That last one opens a real SMTP connection and authenticates — the difference between "a password is set" and "that password works".

If it fails there, it is one of four things, in order of likelihood:

1. The app password was pasted **with spaces** in it
2. The **normal Zoho account password** was pasted instead of the app password
3. `MAIL_HOST` is the wrong region (`smtp.zoho.com` vs `smtp.zoho.in`) or is still unset and defaulting to Gmail
4. Multi-factor auth is not actually on, or the plan does not permit SMTP

6.2 Rehearse with dry run still on

1. Add yourself as a candidate in the Inbox.
2. Draft → Approve → Send.
3. Nothing is delivered; a row appears in the Email Log marked *dry run*.

This proves the whole pipeline — sheet read, template match, merge fields, write-back — with sending physically impossible.

6.3 Go live

1. Settings → **Sending** → on
2. Settings → **dry run** → off
3. **Run preflight again**. The mailbox check is now a hard failure rather than optional.
4. Send one real email **to yourself**.

If it arrives, you are done. Every other send takes the identical code path.

REFERENCE

What protects you once it is live

- **Sending is off by default, twice over** — `dry_run` on *and* `toggle_send` off. Both are deliberate flips.
- **Missing credentials refuse the send** with `E-CONFIG-MISSING` before anything is written. It never logs a send that did not happen.
- **A failed send leaves the row `APPROVED`**, not `SENT`, with the error code in the row. Retry is safe; a candidate cannot be silently lost.
- **Unresolved merge fields block the send** rather than mailing `{{name}}`.
- **Every send lands in the Zoho Sent folder** — a second audit trail, free from using a real mailbox, and the place to check before retrying anything ambiguous.

WHEN SOMETHING BREAKS

CODE	MEANING	FIX
E-SHEET-PERM	Robot cannot see the sheet	Share it with <code>client_email</code> as Editor
E-SHEET-SCHEMA	Columns drifted	<code>npm run bootstrap:sheets</code>
E-CONFIG-MISSING	Dry run off, no mail credentials	Set <code>MAIL_PASSWORD</code> in the environment
E-MAIL-AUTH	Login rejected	Zoho app password (not the account password), no spaces, MFA on, right regional <code>MAIL_HOST</code>
E-MAIL-429	Zoho throttling	Daily send limit for the plan; resumes on its own. Lower <code>send_daily_cap</code>
E-MAIL-REJECTED	Dead address	Fix it in the candidate's Email box
E-MAIL-NETWORK	Cannot reach the server	Check <code>MAIL_HOST</code> region. 465 needs implicit TLS, 587 needs STARTTLS — a mismatch hangs rather than erroring

BROKEN LOGO IN A RECEIVED EMAIL

Not a fault. Mail clients fetch images through their own proxy servers, so a `localhost` URL is unreachable to them and the `alt` text shows instead. Sends from a local machine always look like this. Deployed, `RENDER_EXTERNAL_URL` resolves it automatically and no configuration is needed. To preview locally, point `COMPANY_LOGO_BASE_URL` at the deployed URL — not the `company_logo_url` setting, which lives in the shared sheet and would follow you into production.

ROTATING A SECRET

Same for all of them: revoke at the source, generate a new one, update the environment variable, redeploy. Zoho app password → delete at accounts.zoho.com → Security → App Passwords (<https://accounts.zoho.com>). Service account key → delete under **Keys**. Never edit a secret by committing it.

APPENDIX Deploying for someone else

Only two credentials genuinely require the client's account. The rest you own and reuse across every deployment.

CREDENTIAL	THEIR ACCOUNT?	WHO
MAIL_PASSWORD (Zoho app password) + MAIL_HOST region	Yes — unavoidable	them
SHEET_ID + sheet shared	Yes — one click	them
GOOGLE_SERVICE_ACCOUNT_JSON	No	you
GROQ_API_KEY	No	you
DASHBOARD_PASSWORD	No	you
SESSION_SECRET	No	you

The service account is the non-obvious one: it does not have to live in their Google account. It is a robot address that needs Editor access to their sheet, so you create it once under your own project and they only ever share a spreadsheet with it. That removes an entire Cloud Console walkthrough from the session, leaving about eight minutes of their time.

Have them create the spreadsheet **in their own Drive**. If it sits in yours, their hiring history leaves with you.

Never let an app password travel through chat. Have the deployment environment open and type it straight into the variable while you are with them. If it must be transported, use a password manager's one-time link — never messaging or plain email. And strip the spaces while they can still re-read it: it is shown once, and a wrong paste means booking them again.

COLLECTION SHEET

CLIENT / DATE

CLIENT _____

DATE _____

FROM THEM

MAIL_USER (full Zoho address) _____

MAIL_PASSWORD (Zoho app password, no spaces) _____

MAIL_HOST (region, e.g. smtp.zoho.in) _____

☐ MFA on ☐ plan allows SMTP

SHEET_ID _____

☐ shared with robot as EDITOR

MAIL_FROM (display name) _____

company_email (Reply-To) _____

ALREADY YOURS

☐ GOOGLE_SERVICE_ACCOUNT_JSON

☐ GROQ_API_KEY

☐ DASHBOARD_PASSWORD

☐ SESSION_SECRET

CHECK THE PASSWORD BEFORE THEY WALK AWAY

Ten seconds, sends nothing, and tests the one credential you cannot regenerate without them. Use their regional host.

```
cd dashboard
MAIL_USER=hiring@theircompany.com MAIL_PASSWORD=their-zoho-app-password \
node -e "require('nodemailer').createTransport({host:'smtp.zoho.in',
port:465,secure:true,auth:{user:process.env.MAIL_USER,
pass:process.env.MAIL_PASSWORD}}).verify().then(()=>console.log('works'))
.catch(e=>console.log('FAILED',e.message))"
```

A `535` auth error means the app password is wrong, has spaces, or the host is the wrong region — sort it while they are still in the room.